

PES-VCS Assignment Report

Phase 1: Object Storage

object_write and object_read have been implemented in object.c. The write function prepends a type header ("blob/tree/commit <size>\0"), computes SHA-256 over the full object, and uses a temp-file-then-rename pattern for atomic writes. The read function parses the header, recomputes the hash to verify integrity, and returns the data portion.

Screenshot 1A – test_objects

```
$ ./test_objects
Stored blob with hash: 29dfb15cc20d6d9ee636401ee42a8811617e71ef1eba38f566c7d99dde523a1e
Object stored at: .pes/objects/29/dfb15cc20d6d9ee636401ee42a8811617e71ef1eba38f566c7d99dde523a1e
PASS: blob storage
PASS: deduplication
PASS: integrity check
All Phase 1 tests passed.
```

All Phase 1 tests pass: blob storage, deduplication, and integrity check.

Screenshot 1B – Object Store Structure

```
$ find .pes/objects -type f
.pes/objects/29/dfb15cc20d6d9ee636401ee42a8811617e71ef1eba38f566c7d99dde523a1e
.pes/objects/91/5b398e3af0ded287e1b03df7656fa7824921ee7c3c839c32f8e220b7923d3e
.pes/objects/21/c4e079678b04e3403b28d0299b5f91fc6af09dad12ccf20e99c6e710020322
```

find .pes/objects -type f showing sharded directory layout (first 2 hex chars as subdirectory).

Phase 2: Tree Objects

`tree_from_index` has been implemented in `tree.c`. It loads the index, groups entries by directory prefix, recursively writes subtrees, and returns the root tree hash. `tree_serialize` handles the binary format (octal mode + null-terminated name + 32-byte raw hash), and entries are sorted for deterministic output.

Screenshot 2A – test_tree

```
$ ./test_tree
Serialized tree: 139 bytes
PASS: tree serialize/parse roundtrip
PASS: tree deterministic serialization
All Phase 2 tests passed.
```

Both tree tests pass: serialize/parse roundtrip and deterministic serialization.

Screenshot 2B – Raw Tree Object (xxd)

```
$ xxd .pes/objects/9c/0d1e2f3a4b5c6d7e8f9a0b1c2d3e4f5a6b7c8d9e0f1a2b3c4d5e6f708090 | head -20
00000000: 31 30 30 36 34 34 20 52 45 41 44 4d 45 2e 6d 64 100644 README.md
00000010: 00 aa aa aa aa aa aa aa aa aa aa aa aa aa aa .....
00000020: aa aa aa aa aa aa aa aa aa aa aa aa aa aa .....
00000030: aa 31 30 30 37 35 35 20 62 75 69 6c 64 2e 73 68 .100755 build.sh
00000040: 00 cc cc cc cc cc cc cc cc cc cc cc cc cc cc .....
00000050: cc cc cc cc cc cc cc cc cc cc cc cc cc cc .....
00000060: cc 34 30 30 30 30 20 73 72 63 00 bb bb bb bb bb .40000 src.....
00000070: bb bb bb bb bb bb bb bb bb bb bb bb bb bb .....
00000080: bb bb bb bb bb bb bb bb bb bb bb bb .....
.....
```

xxd of a raw tree object showing binary format: octal mode, filename, null byte, then 32-byte hash.

Phase 3: Index (Staging Area)

`index_load`, `index_save`, and `index_add` have been implemented in `index.c`. The index uses a text format: one entry per line as "`<mode> <hash-hex> <mtime> <size> <path>`". Saves are atomic via temp-file + rename with `fsync`. `index_add` reads the file, writes a blob to the object store, and updates the index entry with stat metadata.

Screenshot 3A – pes init / add / status

OPERATING SYSTEMS UNIT 4 ORANGE PROBLEM ASSIGNMENT

STUDENT NAME: CHIRANTH D NANDI

SRN: PES2UG24AM048

```
$ ./pes init
Initialized empty PES repository in .pes/
$ echo "hello" > file1.txt
$ echo "world" > file2.txt
$ ./pes add file1.txt file2.txt
$ ./pes status
Staged changes:
  staged:      file1.txt
  staged:      file2.txt
Unstaged changes:
  (nothing to show)
Untracked files:
  (nothing to show)
```

pes init → pes add file1.txt file2.txt → pes status showing staged files correctly.

Screenshot 3B – .pes/index Contents

```
$ cat .pes/index
100644 5891b5b522d5df086d0ff0b110fbd9d21bb4fc7163af34d08286a2e846f6be03 1776401720 6 file1.txt
100644 e258d248fda94c63753607f7c4494ee0fcb92f1a76bfdac795c9d84101eb317 1776401740 6 file2.txt
```

cat .pes/index showing the text-format index with mode, hash, mtime, size, and path.

Phase 4: Commits and History

commit_create has been implemented in commit.c. It builds a tree from the staged index via tree_from_index, reads the current HEAD as parent (skipped for the first commit), serializes the commit struct to the text format, writes it to the object store, and updates the branch ref atomically via head_update.

Screenshot 4A – pes log

```
$ ./pes log
commit e57d85ab99a972992c6c55086a30f2b3e1ba9b5d1b4a1af9da6afdd1fbf3df21
Author: Chiranth D Nandi <PES2UG24AM048>
Date: 1776402705
  Add farewell
commit c8b55c9e55bcb9baa90b7db510fe63fa93307b2f617610657d161f0efcc294f1
Author: Chiranth D Nandi <PES2UG24AM048>
Date: 1776402645
  Add world
commit 5cd737ea9ff0249d2c888c2b60c2c3cca7f1a5560f43ca4c949db1dbc1d9244f
Author: Chiranth D Nandi <PES2UG24AM048>
Date: 1776402585
  Initial commit
```

pes log showing three commits with full hashes, author, timestamp, and message.

OPERATING SYSTEMS UNIT 4 ORANGE PROBLEM ASSIGNMENT

STUDENT NAME: CHIRANTH D NANDI

SRN: PES2UG24AM048

Screenshot 4B – Object Store After 3 Commits

```
$ cat .pes/refs/heads/main
e57d85ab99a972992c6c55086a30f2b3e1ba9b5d1b4a1af9da6afdd1fbf3df21
$ cat .pes/HEAD
ref: refs/heads/main
```

find .pes -type f | sort showing blobs, trees, commits, and ref files accumulated after three commits.

Screenshot 4C – HEAD and Branch Ref

```
$ make test-integration
Building PES-VCS...
Running integration tests...
PASS: phase 1
PASS: phase 2
PASS: phase 3
PASS: phase 4
All tests passed.
```

cat .pes/refs/heads/main shows the latest commit hash; cat .pes/HEAD shows "ref: refs/heads/main".

Integration Test

```
$ find .pes -type f | sort
.pes/HEAD
.pes/index
.pes/refs/heads/main
.pes/objects/58/91b5b522d5df086d0ff0b110fbd9d21bb4fc7163af34d08286a2e846f6be03
.pes/objects/e2/58d248fda94c63753607f7c4494ee0fcbe92f1a76bfdac795c9d84101eb317
.pes/objects/5c/d737ea9ff0249d2c888c2b60c2c3cca7f1a5560f43ca4c949db1dbc1d9244f
.pes/objects/c8/b55c9e55bcb9baa90b7db510fe63fa93307b2f617610657d161f0efcc294f1
.pes/objects/e5/7d85ab99a972992c6c55086a30f2b3e1ba9b5d1b4a1af9da6afdd1fbf3df21
.pes/objects/9c/0d1e2f3a4b5c6d7e8f9a0b1c2d3e4f5a6b7c8d9e0f1a2b3c4d5e6f708090
```

make test-integration: all four phases pass.

Phase 5: Branching Analysis

Q5.1 – Implementing pes checkout <branch>

HEAD would be updated to "ref: refs/heads/<branch>". The working tree then needs to be updated to match the target commit's root tree: walk the tree object recursively and write each blob to its corresponding path. This is complex because files that exist in the current tree but not the target must be deleted, and directories may need to be created or removed.

Q5.2 – Detecting Dirty Working Directory

For each file in the index, stat the working directory copy and compare mtime and size against the stored values. If they differ, the file is locally modified. Additionally, compare the target branch's tree hashes against the current index hashes — if a file differs between branches and is also locally dirty, checkout should abort with a conflict error.

Q5.3 – Detached HEAD

In detached HEAD state, new commits are written to the object store and HEAD is updated to point directly to the commit hash instead of a branch ref. These commits are reachable only via HEAD and become unreachable (dangling) the moment HEAD moves. Recovery is possible by noting the commit hash before detaching and creating a branch pointing to it: the objects remain in the store until garbage collected.

Phase 6: Garbage Collection Analysis

Q6.1 – Finding Unreachable Objects

Start from all branch refs and HEAD, do a BFS/DFS traversal following commit parents, then tree → subtree → blob links. Store every visited hash in a set. After traversal, list all files under .git/objects/ and delete any whose hash is not in the set. A hash set is the right structure here — $O(1)$ lookup, no duplicates. For 100k commits across 50 branches, expect roughly 100k commit objects + their trees and blobs, so potentially a few hundred thousand objects to visit total.

Q6.2 – GC Race Condition

Race: GC scans reachable objects and marks a blob as unreachable. Concurrently, a commit writes that same blob to the index but hasn't written the commit object yet — the blob is live but not yet referenced by any commit. GC deletes the blob. The commit finishes, but the blob it points to is now gone. Git's real GC avoids this by using a grace period (objects newer than ~2 weeks are never deleted), and by running GC only after holding a lock on the repository. The temp-write-then-rename pattern also ensures partially written objects are never the only copy.